



Rol del modelo

Cuando llamas a un LLM, no solo envías una pregunta: envías **instrucciones** que condicionan cómo responde. Una de las más potentes es el **rol**: quién debe “actuar” el modelo al responder.

No es magia. No cambias el modelo. Cambias las instrucciones.

La llamada a un mismo modelo, con la misma pregunta base, puede devolver tonos, profundidades y vocabulario muy distintos si cambiamos el rol.

Objetivos

- Entender qué es un **rol** en un prompt y qué no es.
 - Diferenciar **pregunta suelta** vs **instrucción con rol**.
 - Usar conceptos: rol, objetivo, audiencia, restricciones.
 - Construir prompts con rol desde Python y comparar salidas con Gemini.
-

1) ¿Qué es un rol?

El **rol** define la identidad o perspectiva que el modelo debe adoptar:

- Profesor universitario vs profesor de primaria.
- Abogado vs médico vs desarrollador senior.
- Entrevistador de RR. HH. vs redactor de marketing.

No cambia los pesos del modelo. Cambia el **marco** en el que genera texto: estilo, nivel, prioridades, vocabulario.

En APIs como Gemini, el rol suele ir en el mismo **contents** que el resto del prompt (instrucciones + datos). Más adelante algunos sistemas separan “system” y “user”; el concepto pedagógico es el mismo: **quién eres al responder**.

2) Instrucción vs pregunta

Tipo	Ejemplo	Qué suele pasar
Pregunta suelta	¿Qué es una API?	Respuesta genérica, estilo enciclopedia
Instrucción con rol	Eres un profesor universitario. Explica qué es una API.	Más estructura académica, definiciones, rigor
Rol + audiencia	Eres profesor de programación para niños de 12 años. Explica qué es una API.	Lenguaje simple, analogías, menos jerga

La pregunta aporta el **tema**. El rol aporta **cómo** tratar el tema.

Buena práctica: en aplicaciones reales, casi nunca envíes solo la pregunta del usuario; envía **rol + tarea + datos + formato** (ver documentos 02 y 06).

3) Conceptos que suelen acompañar al rol

Rol

Quién eres: Eres un analista de soporte técnico con 10 años de experiencia.

Objetivo

Para qué respondes (puede solaparse con la "tarea" del doc 02): Tu objetivo es ayudar al usuario a entender el error sin tecnicismos innecesarios.

Audiencia

A quién te diriges: El lector es un alumno que nunca ha programado.

Restricciones de estilo o contenido

- No uses más de 150 palabras.
- No inventes datos; si no sabes algo, dilo.
- Responde en español de España.

El rol sin audiencia ni restricciones sigue siendo útil; con ellos, el control es mayor.

4) El mismo modelo, la misma pregunta, resultados distintos

Experimento conceptual (tres llamadas, mismo `model`, distinto `contents`):

Pregunta base: ¿Qué es una API?

Caso 1 — sin rol

```
prompt = "¿Qué es una API?"
```

Caso 2 — rol académico

```
prompt = """
Eres un profesor universitario de ingeniería informática.
Explica qué es una API de forma clara y estructurada.
"""
```

Caso 3 — rol + audiencia infantil

```
prompt = """
Eres un profesor de programación para niños de 12 años.
Explica qué es una API con un ejemplo cotidiano y lenguaje sencillo.
Evita siglas sin explicarlas.
"""
```

Aprendizaje: el alumno debe **ejecutar** los tres y comparar longitud, tono y estructura. Eso demuestra el “control” sin tocar código del proveedor.

5) Roles en Python: diccionario reutilizable

Patrón habitual: roles en un `dict` y selección por clave (menú, config o argumento).

```
ROLES = {
    "teacher": (
        "Eres un profesor de programación paciente y claro. ",
        "Usas ejemplos cortos."
    ),
    "lawyer": (
        "Eres un abogado que explica conceptos con precisión. ",
        "Evitas consejos legales vinculantes; solo educación general."
    ),
    "doctor": (
        "Eres un médico que explica salud en lenguaje accesible. ",
        "No diagnosticas ni prescribes; recomiendas consultar a un profesional."
    ),
    "scrum_master": (
        "Eres un Scrum Master. ",
        "Enfocas la respuesta en equipo, iteración y mejora continua."
    ),
    "interviewer": (
        "Eres un entrevistador técnico. ",
        "Haces preguntas de seguimiento y pides ejemplos concretos."
    ),
    "marketing": (
        "Eres un redactor de marketing digital. ",
        "Respuestas persuasivas pero honestas, sin exagerar."
    ),
}
```

```

}

def build_prompt_con_rol(role_key: str, pregunta: str) -> str:
    if role_key not in ROLES:
        raise ValueError(f"Rol desconocido: {role_key}. Opciones: {list(ROLES)}")
    rol = ROLES[role_key]
    return f"{rol.strip()}\n\nPregunta del usuario:\n{pregunta.strip()}\n"

```

Dónde vive en un proyecto: `prompts.py` o `config.py` (solo textos) según prefieras (Unidad 1).

6) Llamada completa a Gemini

```

import os
from google import genai

client = genai.Client() # GEMINI_API_KEY en el entorno

pregunta = "¿Qué es una API?"
role_key = "teacher"

prompt = build_prompt_con_rol(role_key, pregunta)

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=prompt,
)

print(f"--- Rol: {role_key} ---")
print(response.text)

```

Bucle para comparar roles:

```

for key in ("teacher", "lawyer", "marketing"):
    prompt = build_prompt_con_rol(key, pregunta)
    response = client.models.generate_content(
        model="gemini-3-flash-preview",
        contents=prompt,
    )
    print(f"\n=== {key} ===\n{response.text[:500]}\n")

```

(Truncar en consola solo para leer; en clase conviene ver respuestas completas de 2–3 roles.)

7) Rol del usuario vs rol del modelo

No confundir:

- **Rol del modelo:** “Eres un profesor...” (lo que tú pones en el prompt).
- **Rol del usuario en un chat:** en APIs multi-turno, mensajes **user** / **model**; eso es conversación, no necesariamente “rol profesional”.

En esta unidad, “rol” = **instrucción de comportamiento del asistente**.

8) Límites del rol

El rol **orienta** al modelo; no **garantiza**:

- Veracidad factual (un “médico” puede alucinar).
- Cumplimiento legal o ético (siempre revisa contenido sensible).
- Igualdad de estilo en todas las ejecuciones (hay variabilidad).

Por eso en sistemas serios combinas rol con:

- tareas específicas (doc 02),
 - formato de salida (doc 06),
 - validación del output (Unidad 1).
-

9) Buenas prácticas (rol)

1. **Un rol principal por llamada** — evita “eres profesor y abogado y chef...” salvo que la tarea lo exija.
 2. **Rol estable en prompts.py** — el usuario no reescribe el rol cada vez.
 3. **Audiencia explícita** si el nivel importa (principiante vs experto).
 4. **Prohibiciones claras** cuando el dominio es sensible (salud, legal).
 5. **Mismo rol + distintas tareas** — el rol cambia poco; la tarea cambia mucho (siguiente documento).
-

10) Errores frecuentes

Error	Mejor enfoque
Solo pegar la pregunta del usuario	Prefijar rol + tarea
Rol demasiado largo y vago	2–4 frases concretas
Mezclar rol y tarea en un párrafo ilegible	Secciones: ROL: , TAREA:
Esperar que el rol corrija datos falsos	Validar hechos; no confiar ciegamente

11) Puente con el siguiente tema

- **Rol** = *quién habla y con qué estilo*.
- **Tarea** (doc 02) = *qué debe entregar y con qué estructura*.

Ejemplo combinado (adelanto):

```
[Eres un profesor de programación para principiantes.] ← rol
```

Analiza el siguiente texto y devuelve:

- tema principal
 - sentimiento (positivo / neutro / negativo)
 - resumen en una frase
- ← tarea

Resumen

- El **rol** condiciona tono, nivel y enfoque; no cambia el modelo subyacente.
- **Pregunta ≠ instrucción completa**; en productos reales casi siempre incluyes rol (y más).
- En Python: **ROLES en dict** + `build_prompt_con_rol` + llamada Gemini.
- Comparar salidas con el mismo modelo es la demostración pedagógica clave.

Siguiente lectura: [02 — Definición de tareas](#).